

SKILL SESSION 5&6

Session 5:

TASK 1:

```
[(base) saiabburi@Geyasree-MacBook-Air ~ % mkdir Skill3
[(base) saiabburi@Geyasree-MacBook-Air ~ % cd Skill3
[(base) saiabburi@Geyasree-MacBook-Air Skill3 % name="Charishma"
[(base) saiabburi@Geyasree-MacBook-Air Skill3 % echo $name
Charishma
[(base) saiabburi@Geyasree-MacBook-Air Skill3 % echo "My name is $name"
My name is Charishma
[(base) saiabburi@Geyasree-MacBook-Air Skill3 % echo 'My name is $name'
My name is $name
[(base) saiabburi@Geyasree-MacBook-Air Skill3 % message='Hello $name, this is a test'
(base) saiabburi@Geyasree-MacBook-Air Skill3 % echo "$message"
Hello $name, this is a test
(base) saiabburi@Geyasree-MacBook-Air Skill3 % echo 'Hello * ? $ # @ !'
Hello * ? $ # @ !
(base) saiabburi@Geyasree-MacBook-Air Skill3 % echo 'This is a sentence with many spaces'
This is a sentence with many spaces
(base) saiabburi@Geyasree-MacBook-Air Skill3 % echo 'He said "Hello" to me'
He said "Hello" to me
```

TASK 2:

```
[(base) saiabburi@Geyasree-MacBook-Air Skill3 % name="Charishma"
course="Computer Science"
(base) saiabburi@Geyasree-MacBook-Air Skill3 % echo "My name is $name"
My name is Charishma
[(base) saiabburi@Geyasree-MacBook-Air Skill3 % echo "I study $course"
I study Computer Science
(base) saiabburi@Geyasree-MacBook-Air Skill3 % echo My name is Charishma
My name is Charishma
[(base) saiabburi@Geyasree-MacBook-Air Skill3 % college="My College"
(base) saiabburi@Geyasree-MacBook-Air Skill3 % echo "$college"
My College
(base) saiabburi@Geyasree-MacBook-Air Skill3 % echo $college
My College
(base) saiabburi@Geyasree-MacBook-Air Skill3 % echo "$college"
My College
(base) saiabburi@Geyasree-MacBook-Air Skill3 % set -- $college
echo $#
1
[(base) saiabburi@Geyasree-MacBook-Air Skill3 % set -- "$college"
echo $#
1
[(base) saiabburi@Geyasree-MacBook-Air Skill3 % echo "Hello $name"
Hello Charishma
[(base) saiabburi@Geyasree-MacBook-Air Skill3 % echo 'She said "Hello" to me'
She said "Hello" to me
[(base) saiabburi@Geyasree-MacBook-Air Skill3 %
```

Session 6:

TASK 1:

```
[(base) saiabburi@Geyasree-MacBook-Air Skill3 % echo Hello\ World
Hello World
[(base) saiabburi@Geyasree-MacBook-Air Skill3 % echo Hello\!
Hello!
[(base) saiabburi@Geyasree-MacBook-Air Skill3 % touch My\ File.txt
[(base) saiabburi@Geyasree-MacBook-Air Skill3 % ls
My File.txt
[(base) saiabburi@Geyasree-MacBook-Air Skill3 % echo "Testing file" > My\ File.txt
[(base) saiabburi@Geyasree-MacBook-Air Skill3 % cat My\ File.txt
Testing file
[(base) saiabburi@Geyasree-MacBook-Air Skill3 % echo \$name
$name
[(base) saiabburi@Geyasree-MacBook-Air Skill3 % echo "\"Hello\""
"Hello"
[(base) saiabburi@Geyasree-MacBook-Air Skill3 % echo Hello\ World\!\ $name\ \#Test
Hello World! $name #Test
```

TASK 2:

```
[(base) saiabburi@Geyasree-MacBook-Air Skill3 % nano child.c
[(base) saiabburi@Geyasree-MacBook-Air Skill3 % clang child.c -o child
[(base) saiabburi@Geyasree-MacBook-Air Skill3 % ls
My File.txt      child      child.c
[(base) saiabburi@Geyasree-MacBook-Air Skill3 % ./child
Parent process started. PID = 5527
Parent waiting for child...
Child process created. PID = 5537
Child executing echo command...
Hello from child process
Child completed with exit status 0
Parent process completed.
```

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid;
    int status;

    printf("Parent process started. PID = %d\n", getpid());

    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    if (pid == 0) {
        printf("Child process created. PID = %d\n", getpid());

        char *args[] = {"echo", "Hello from child process", NULL};

        printf("Child executing echo command...\n");

        execvp(args[0], args);

        perror("execvp failed");
        exit(1);
    }

    else {
        printf("Parent waiting for child...\n");

        waitpid(pid, &status, 0);

        if (WIFEXITED(status)) {
            printf("Child completed with exit status %d\n",
                WEXITSTATUS(status));
        }

        printf("Parent process completed.\n");
    }

    return 0;
}
```

```
[(base) saiabburi@Geyasree-MacBook-Air Skill3 % nano child.c
[(base) saiabburi@Geyasree-MacBook-Air Skill3 %
[(base) saiabburi@Geyasree-MacBook-Air Skill3 % clang child.c -o child
[(base) saiabburi@Geyasree-MacBook-Air Skill3 % ./child
Parent process started. PID = 5550
Parent waiting for child...
Child process created. PID = 5551
Child executing echo command...
Hello Charishma from child
Child completed with exit status 0
Parent process completed.
[(base) saiabburi@Geyasree-MacBook-Air Skill3 % ls
My File.txt      child      child.c
```

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid;
    int status;

    printf("Parent process started. PID = %d\n", getpid());

    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    if (pid == 0) {
        printf("Child process created. PID = %d\n", getpid());

        char *args[] = {"echo", "Hello", "Charishma", "from", "child", NULL};

        printf("Child executing echo command...\n");

        execvp(args[0], args);

        perror("execvp failed");
        exit(1);
    }

    else {
        printf("Parent waiting for child...\n");

        waitpid(pid, &status, 0);

        if (WIFEXITED(status)) {
            printf("Child completed with exit status %d\n",
                WEXITSTATUS(status));
        }

        printf("Parent process completed.\n");
    }

    return 0;
}
```

```
(base) saiabburi@Geyasree-MacBook-Air Skill3 % nano child.c
(base) saiabburi@Geyasree-MacBook-Air Skill3 % clang child.c -o child
(base) saiabburi@Geyasree-MacBook-Air Skill3 % ./child
Parent process started. PID = 5611
Parent waiting for child...
Child process created. PID = 5612
Child executing echo command...
execvp failed: No such file or directory
Child completed with exit status 1
Parent process completed.
(base) saiabburi@Geyasree-MacBook-Air Skill3 %
```

PRACTICE PROBLEMS:

TASK 1: To Create Main Loop, Display Prompt, Read User Input, Handle Exit Conditions, Design Control Flow Diagram, Test Interactive Loop.

```
[(base) saiabburi@Geyasree-MacBook-Air Skill3 % clang task1.c -o task1
[(base) saiabburi@Geyasree-MacBook-Air Skill3 % ./task1
myshell> Helloo
You entered: Helloo
myshell> Testt
You entered: Testt
myshell> Hello World this is charishma
You entered: Hello World this is charishma
myshell> exit
Exiting shell...
(base) saiabburi@Geyasree-MacBook-Air Skill3 %
```

UW PICO 5.09

File: task1.c

```
#include <stdio.h>
#include <string.h>

int main() {
    char input[100];

    while (1) {
        printf("myshell> ");
        fgets(input, sizeof(input), stdin);

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0) {
            printf("Exiting shell...\n");
            break;
        }

        if (strlen(input) == 0) {
            continue;
        }

        printf("You entered: %s\n", input);
    }

    return 0;
}
```

TASK 2: Using the following system calls, copy the content from File1.txt to File2.txt.

1. Open()
2. Read()
3. Write()
4. Close()

```
[(base) saiabburi@Geyasree-MacBook-Air Skill3 % echo "This is the content of File1." > File1.txt
[(base) saiabburi@Geyasree-MacBook-Air Skill3 % cat File1.txt
This is the content of File1.
[(base) saiabburi@Geyasree-MacBook-Air Skill3 % nano task2.c
[(base) saiabburi@Geyasree-MacBook-Air Skill3 % clang task2.c -o task2
[(base) saiabburi@Geyasree-MacBook-Air Skill3 % ./task2
File copied successfully.
[(base) saiabburi@Geyasree-MacBook-Air Skill3 % cat File2.txt
This is the content of File1.
[(base) saiabburi@Geyasree-MacBook-Air Skill3 % LS
File1.txt      My File.txt    child.c        task1.c        task2.c
File2.txt      child          task1          task2
[(base) saiabburi@Geyasree-MacBook-Air Skill3 %
```

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>

int main() {
    int source, destination;
    char buffer[100];
    ssize_t bytesRead, bytesWritten;

    source = open("File1.txt", O_RDONLY);

    if (source < 0) {
        perror("Error opening File1.txt");
        return 1;
    }

    destination = open("File2.txt",
                      O_WRONLY | O_CREAT | O_TRUNC,
                      0644);

    if (destination < 0) {
        perror("Error opening File2.txt");
        close(source);
        return 1;
    }

    while ((bytesRead = read(source, buffer, sizeof(buffer))) > 0) {
        bytesWritten = write(destination, buffer, bytesRead);

        if (bytesWritten != bytesRead) {
            perror("Error writing to File2.txt");
            close(source);
            close(destination);
            return 1;
        }
    }

    close(source);
    close(destination);

    printf("File copied successfully.\n");

    return 0;
}
```

TASK 3: To Capture Keyboard Input, Handle Backspace, Process Enter Key, Manage Input Buffer, Support Multi-Character Commands, Test User Interaction.

```
File2.txt      Child      task1      task2
(base) saiabburi@Geyasree-MacBook-Air Skill3 % nano task3.c
(base) saiabburi@Geyasree-MacBook-Air Skill3 % clang task3.c -o task3
(base) saiabburi@Geyasree-MacBook-Air Skill3 % ./task3
Enter command: Helloo
Helloo
You entered: Helloo
(base) saiabburi@Geyasree-MacBook-Air Skill3 %
```

```
UW PICO 5.09                                     File: task3.c

#include <stdio.h>
#include <string.h>

int main() {
    char buffer[100];
    int index = 0;
    int ch;

    printf("Enter command: ");

    while ((ch = getchar()) != '\n') {
        if (ch == 127 || ch == 8) {
            if (index > 0) {
                index--;
                printf("\b \b");
                fflush(stdout);
            }
        }
        else if (index < 99) {
            buffer[index++] = ch;
            putchar(ch);
            fflush(stdout);
        }
    }

    buffer[index] = '\0';

    printf("\nYou entered: %s\n", buffer);

    return 0;
}
```

TASK 4: To Apply Escape Sequences, Store Command History, Navigate Previous Commands, Navigate Next Commands, Update Input Buffer, Test Recall Functionality.

```
[(base) saiabburi@Geyasree-MacBook-Air Skill3 % nano task4.c
[(base) saiabburi@Geyasree-MacBook-Air Skill3 % clang task4.c -o task4
[(base) saiabburi@Geyasree-MacBook-Air Skill3 % ./task4
Simple Command History
Type 'history' to view commands.
Type 'exit' to quit.

myshell> ls
Command stored: ls
myshell> history

Command History:
1: ls
myshell> pwd
Command stored: pwd
myshell> history

Command History:
1: ls
2: pwd
myshell> exit
Shell terminated.
(base) saiabburi@Geyasree-MacBook-Air Skill3 %
```

```
#include <stdio.h>
#include <string.h>

#define MAX_HISTORY 10
#define MAX_LENGTH 100

int main() {
    char history[MAX_HISTORY][MAX_LENGTH];
    char input[MAX_LENGTH];
    int count = 0;

    printf("Simple Command History\n");
    printf("Type 'history' to view commands.\n");
    printf("Type 'exit' to quit.\n\n");

    while (1) {
        printf("myshell> ");
        fgets(input, MAX_LENGTH, stdin);

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0) {
            break;
        }

        if (strcmp(input, "history") == 0) {
            printf("\nCommand History:\n");

            for (int i = 0; i < count; i++) {
                printf("%d: %s\n", i + 1, history[i]);
            }

            continue;
        }
    }
```

```
        if (strcmp(input, "history") == 0) {
            printf("\nCommand History:\n");

            for (int i = 0; i < count; i++) {
                printf("%d: %s\n", i + 1, history[i]);
            }

            continue;
        }

        if (strlen(input) > 0) {
            if (count < MAX_HISTORY) {
                strcpy(history[count], input);
                count++;
            } else {
                for (int i = 0; i < MAX_HISTORY - 1; i++) {
                    strcpy(history[i], history[i + 1]);
                }

                strcpy(history[MAX_HISTORY - 1], input);
            }

            printf("Command stored: %s\n", input);
        }
    }

    printf("Shell terminated.\n");

    return 0;
}
```

TASK 5: To Allocate Buffers Dynamically, Resize Arrays, Prevent Buffer Overflow, Manage Linked Lists, Release Memory Correctly, Verify with Valgrind.

Validate the program with valgrind tool.

```
Shell terminated.
(base) saiabburi@Geyasree-MacBook-Air Skill3 % nano task5.c
(base) saiabburi@Geyasree-MacBook-Air Skill3 % clang task5.c -o task5
(base) saiabburi@Geyasree-MacBook-Air Skill3 % ./task5
Enter text: This is a very long sentence used to test dynamic buffer resizing.
Stored text: This is a very long sentence used to test dynamic buffer resizing.
Buffer size: 80
Linked list data: This is a very long sentence used to test dynamic buffer resizing.
Memory released successfully.
(base) saiabburi@Geyasree-MacBook-Air Skill3 %
```

```
UW PICO 5.09 File: task5.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct Node {
    char *data;
    struct Node *next;
} Node;

int main() {
    int capacity = 10;
    int size = 0;

    char *buffer = malloc(capacity);

    if (buffer == NULL) {
        perror("malloc failed");
        return 1;
    }

    printf("Enter text: ");
    int ch;

    while ((ch = getchar()) != '\n' && ch != EOF) {
        if (size >= capacity - 1) {
            capacity *= 2;

            char *temp = realloc(buffer, capacity);

            if (temp == NULL) {
                free(buffer);
                perror("realloc failed");
                return 1;
            }

            buffer = temp;
        }

        buffer[size++] = ch;
    }

    buffer[size] = '\0';
```

```
printf("Stored text: %s\n", buffer);
printf("Buffer size: %d\n", capacity);

Node *head = malloc(sizeof(Node));

if (head == NULL) {
    free(buffer);
    return 1;
}

head->data = malloc(strlen(buffer) + 1);

if (head->data == NULL) {
    free(head);
    free(buffer);
    return 1;
}

strcpy(head->data, buffer);
head->next = NULL;

printf("Linked list data: %s\n", head->data);

free(head->data);
free(head);
free(buffer);
```